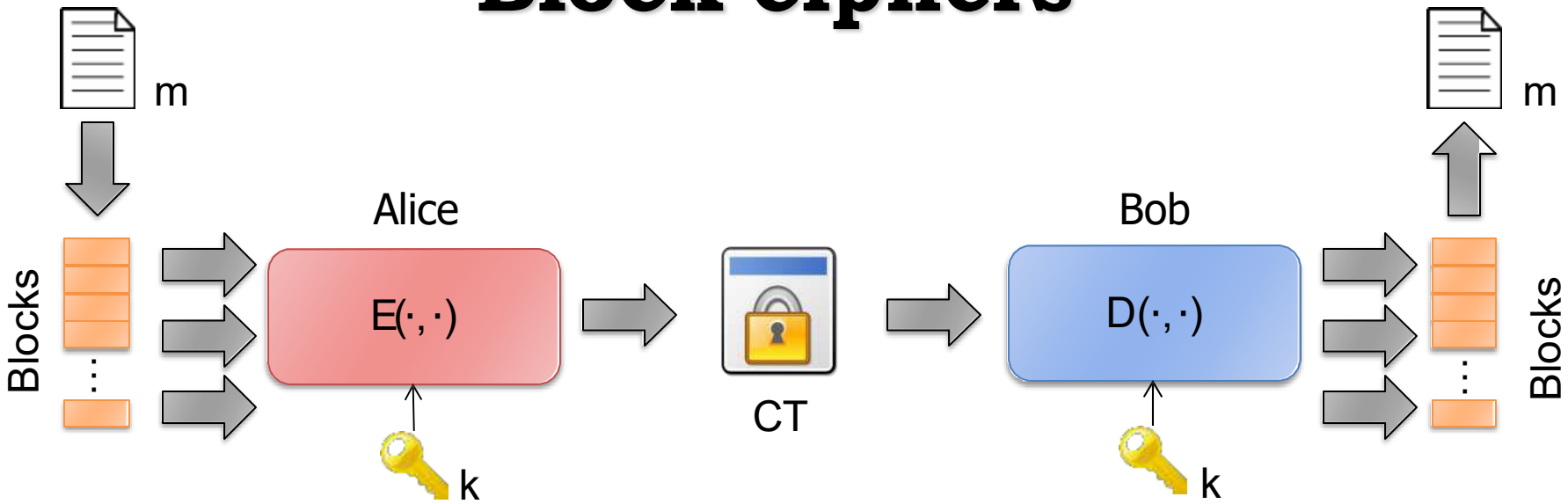
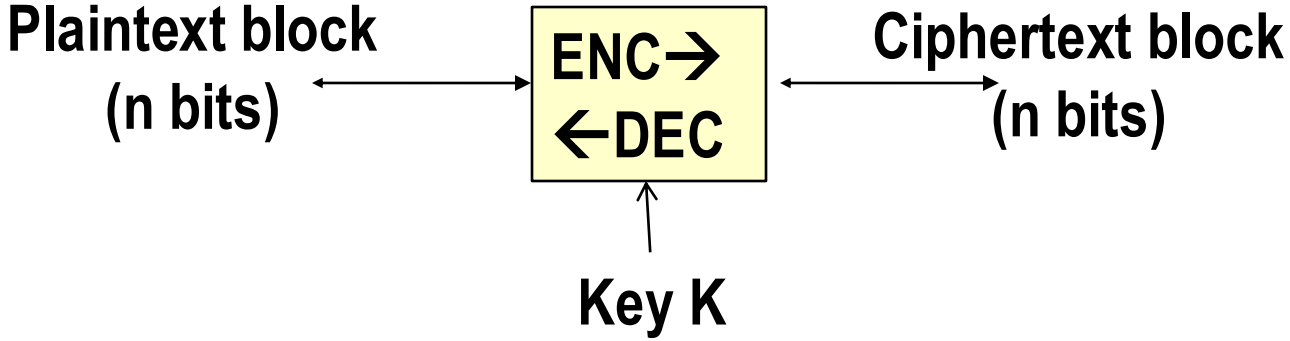
Review of Block cipher Modes of Operation

Block ciphers



➔**Goal: "generalize" substitution ciphers**



➔ **Block algorithm should implement a Pseudo Random Permutation**

⇨ *in practice key will permit to select "only" among 2^(keysize) permutations*

Graphics courtesy of Marco Spaziani Brunella

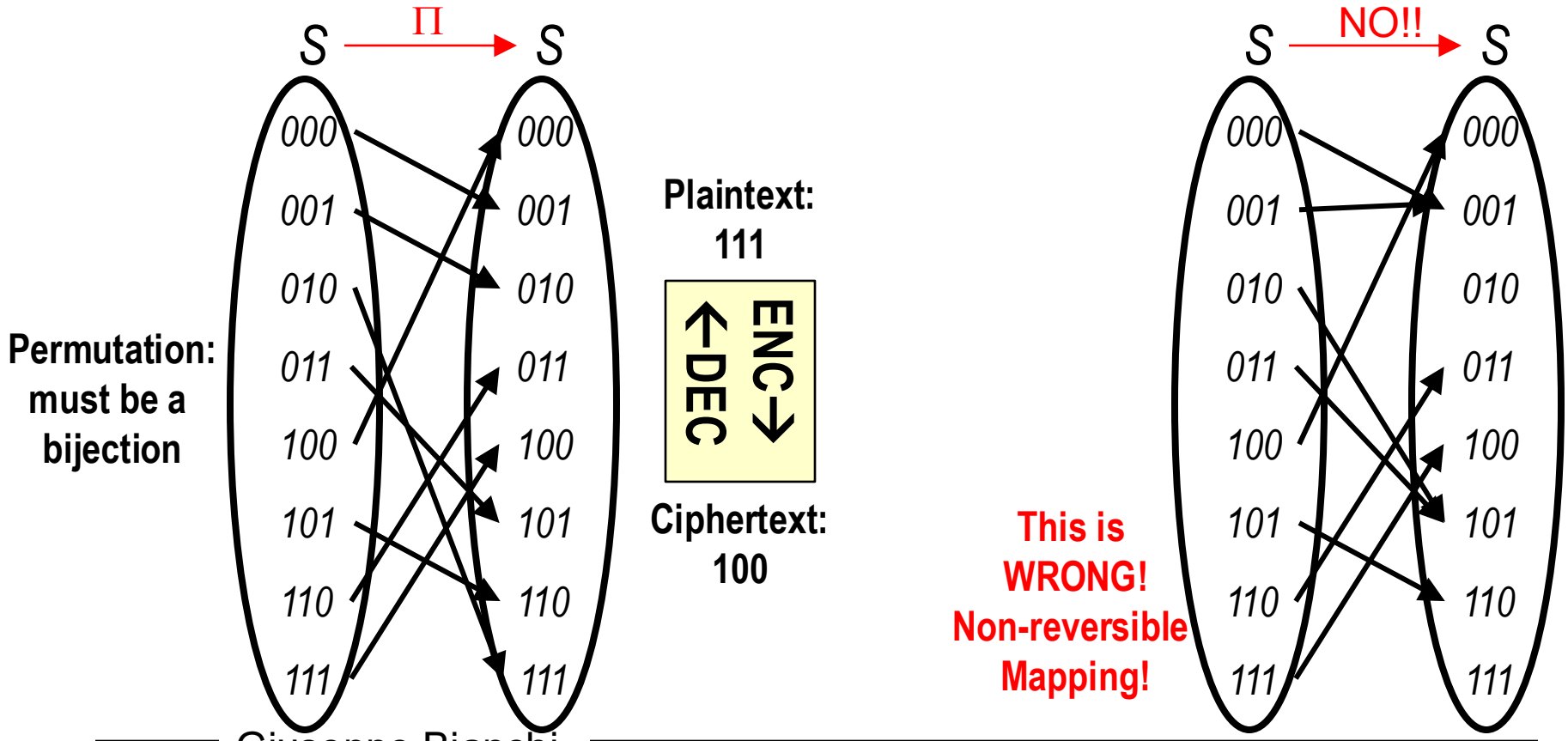
Block ciphers → PRPs

Pseudo Random Permutation:

→ S = set of all possible plaintexts

→n = 3 bits, S = {000,001,010,011,100,101,110,111}, |S| = $2^n = 8$

→ Permutation: bijective function Π: S→S (1-to-1 mapping)



Block ciphers → PRPs

Pseudo Random Permutation:

→ S = set of all possible plaintexts

→ n = 3 bits, S = {000,001,010,011,100,101,110,111}, |S| = $2^n = 8$

→ Permutation: bijective function Π: S→S (1-to-1 mapping)

→ Pseudo-Random: block cipher should uniformly select one of the possible permutations

⇨ *Permutation Π_K used by the cipher is «selected» by the secret key K*

→ PROBLEM!! How many permutations? 2^n! (! = factorial)

⇨ *For n=3: $2^3! = 8! = 40320$*

⇨ *For n=8: $2^8! = 256! = 8.58 \times 10^{506}$ =*

*85781777534284265411908227168123262515778152027948561985965565037726945255314758937744029136045140845037588534233658430615719683469369647532228928849742602567963733256336878644267520762679456018796886797152114330770207752664645146470918732610083287632570281898077367178145417025052301860849531906813825748107025281755945947698703466571273813928620523475680821886070120361108315209350194743710910172696826286160626366243502284094419140842461593600000000000

Block ciphers → PRPs

Pseudo Random Permutation:

→ S = set of all possible plaintexts

→n = 3 bits, S = {000,001,010,011,100,101,110,111}, $|S| = 2^n = 8$

How to compute large factorials?
Stirling Approximation

$$n! \approx \sqrt{2\pi n}\frac{n^n}{e^n}$$

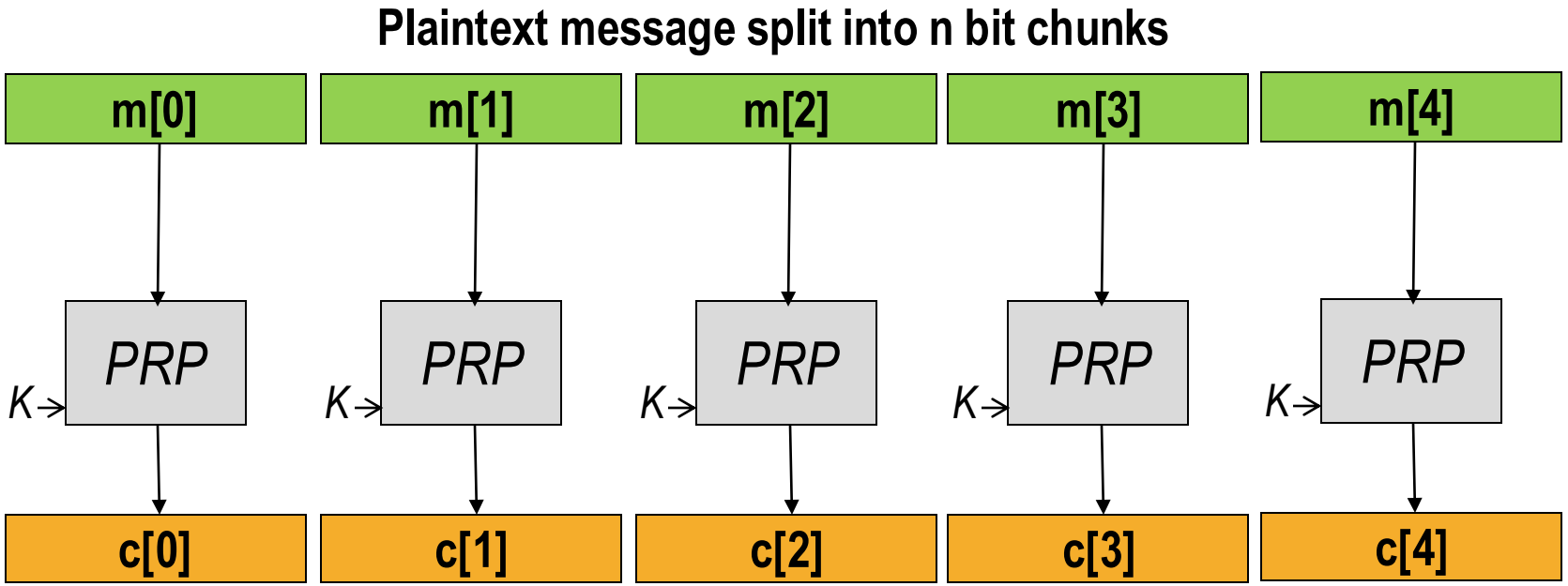
$$2^{128}! \approx \sqrt{2\pi 2^{128}} \cdot 2^{128 \cdot 2^{128}} / e^{2^{128}} \approx$$

$$\approx \sqrt{2\pi} \cdot 2^{64+2^{135}-2^{128}\cdot \log_2 e} \approx 2^{2^{135}}$$

→AES keys: 128, 192, 256 → 2^128 << 2^135 ! (2^135 just 78 digits key)

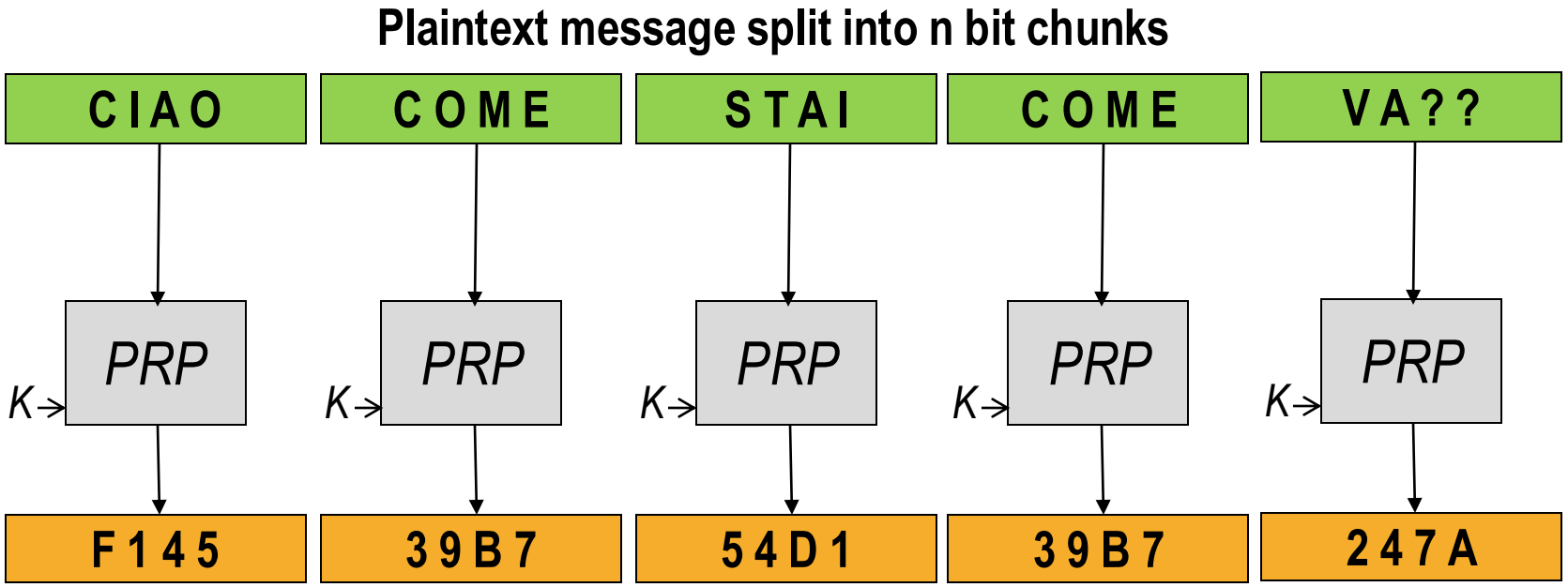
⇨ *Take home: Actual AES permutations way less that ideal PRP (but still OK)*

Problem 1
Plaintext longer than block size?



Independent encryption of each block = Electronic Code Book (ECB Mode)

Problem 1
Plaintext longer than block size?



Independent encryption of each block = Electronic Code Book (ECB Mode)

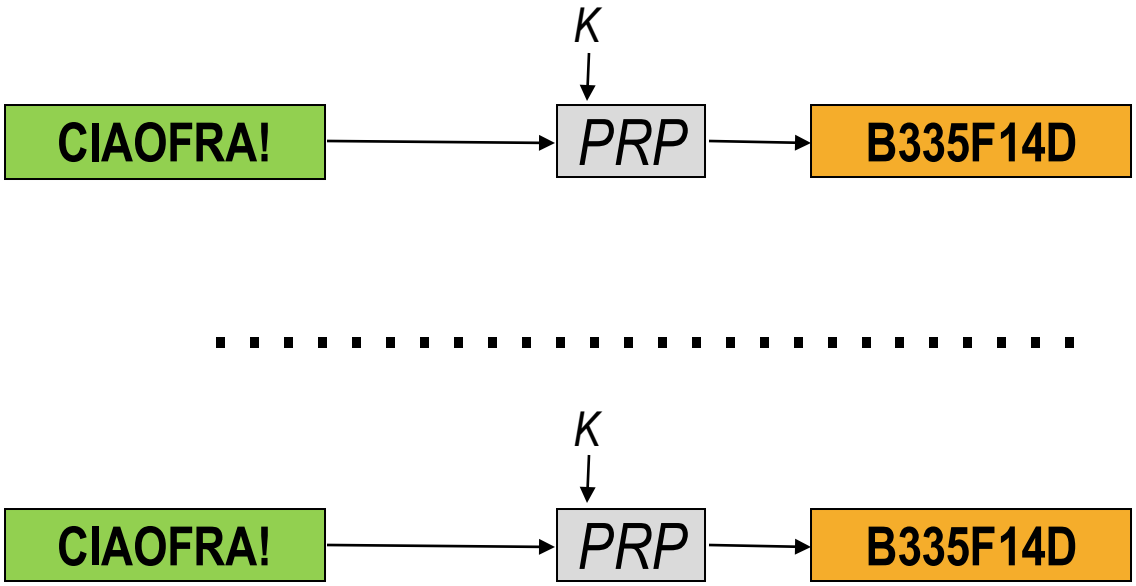
Same plaintext block → Same ciphertext block!!!

No semantic security → Trivial Cryptoanalysis

REMEMBER: never, NEVER use ECB!!!



Problem 2
Encrypt same message twice?

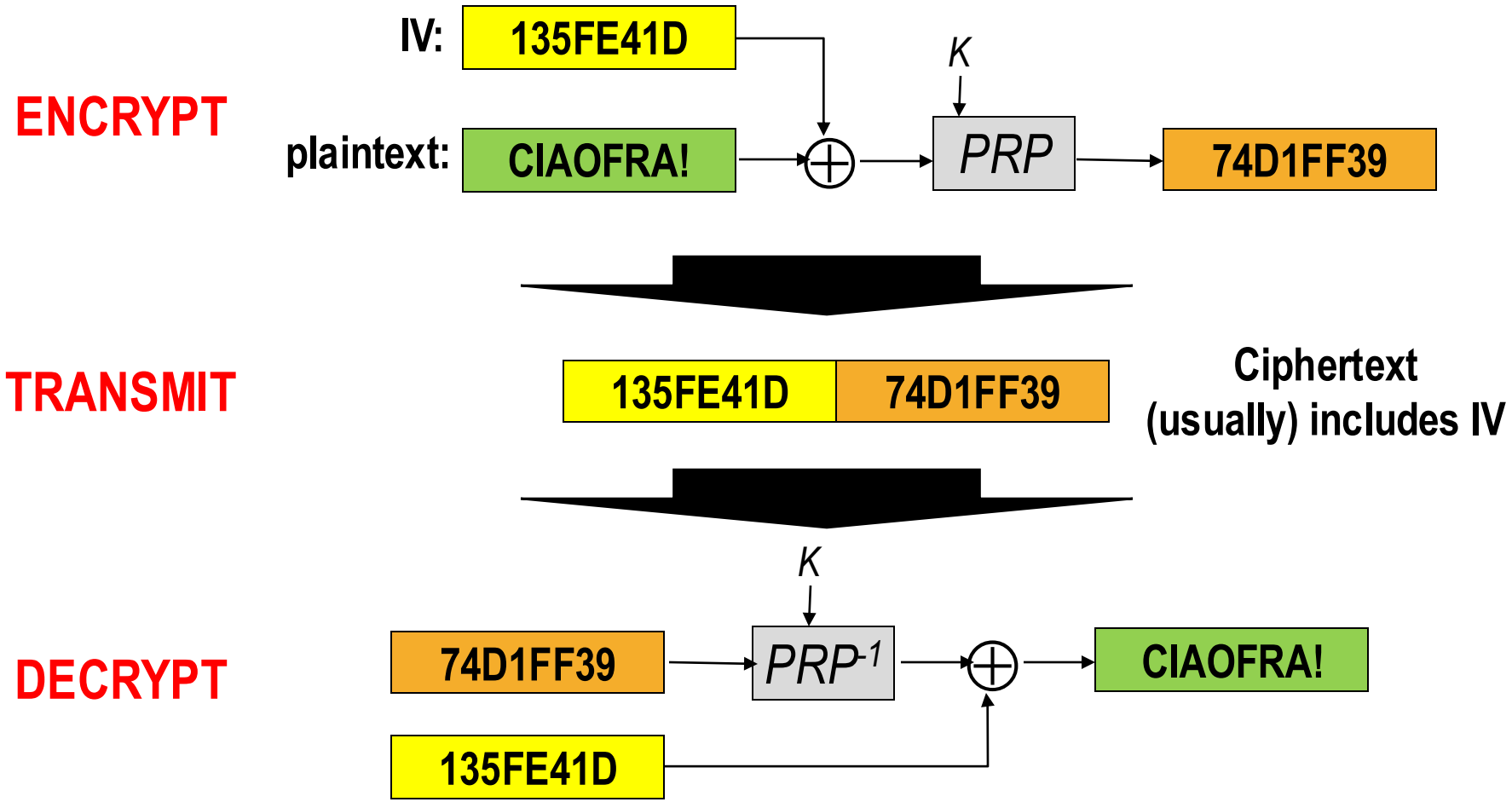


CT = ENC(PT, key) → same key, same PT = same CT!

Same problem as in stream ciphers → same idea!

A fresh Initialization Vector (IV) for every new encryption!

Initialization Vectors in block ciphers



IV NEVER REPEATS? YES BUT not enough! IV must now ALSO be UNpredictable (why? more later)

Block ciphers: modes of operation

➔ **Plain ECB is only OK if**

- ⇨ *Message smaller than one block*
  - →AES = 128 bits = 16 bytes, very rare as short
- ⇨ *Message will never repeat*

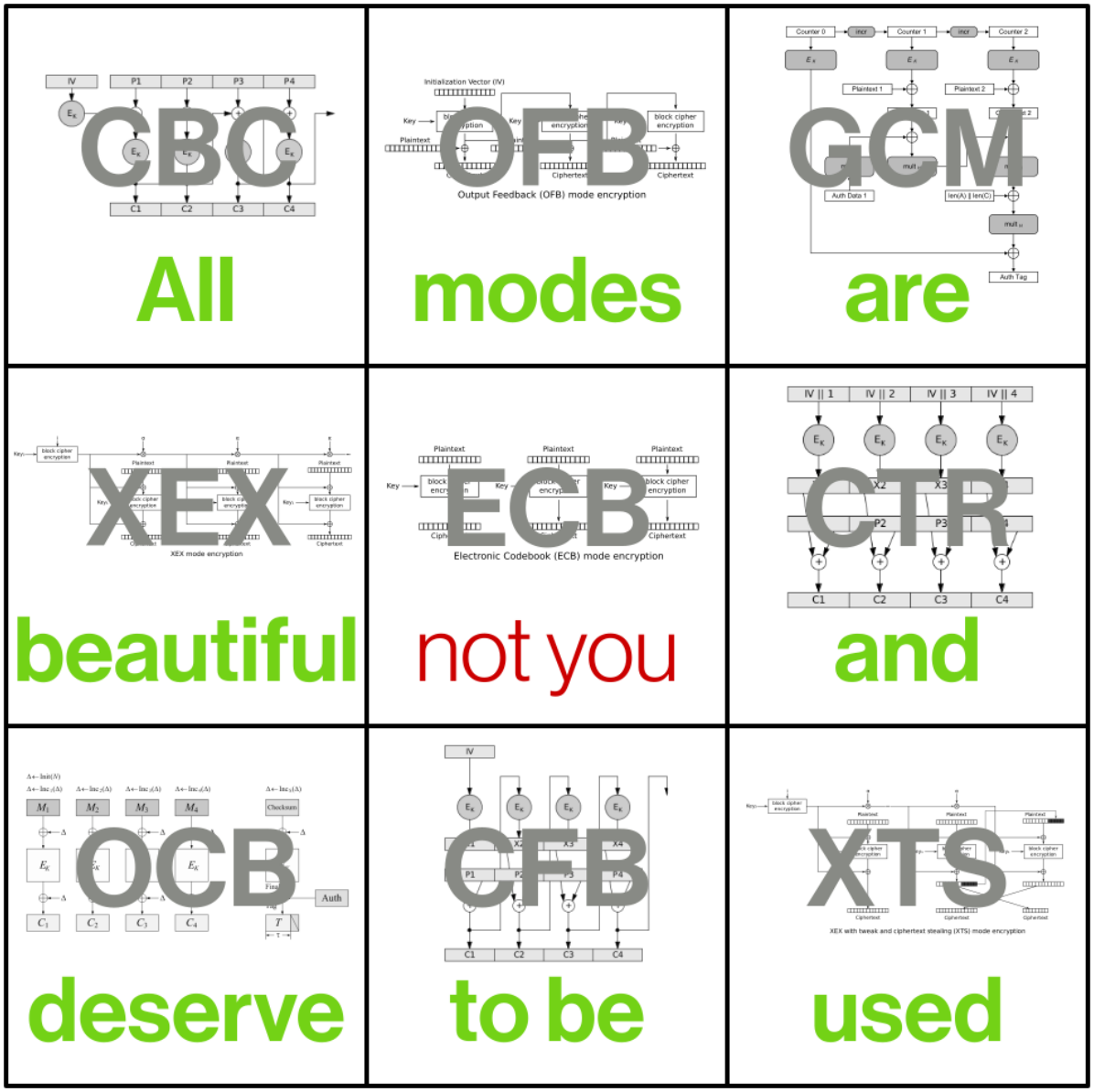
➔ **For repeated messages:**

- ⇨ *Use initialization vector*
  - →Random value, same size as block

➔ **For longer messages:**

- ⇨ **Modes of operation***: "secure" way to combine blocks*
- ⇨ *Permits to encrypt a text of arbitrary length,* **avoiding that a same plaintext block is encrypted twice as same ciphertext**
- ⇨ *"secure" = semantic security, IND-CPA*

Block ciphers: modes of operation

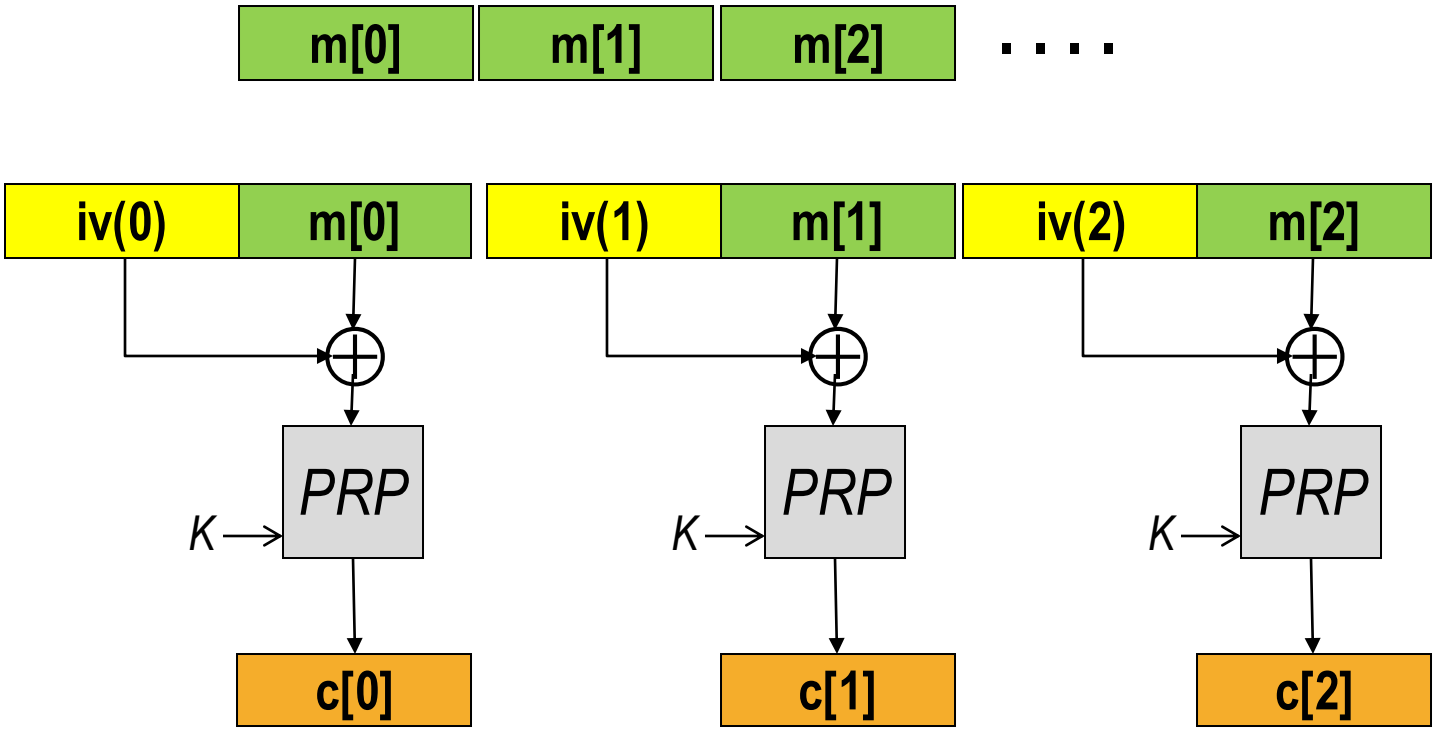


Most used:
Cipher Block Chaining (CBC)
Counter Mode (CTR)

NIST recommended (in 2001 document) but not much used in practice – despite quite interesting properties
Cipher Feedback Mode (CFB)
Output Feedback Mode (OFB)

More advanced properties (authenticated encryption)
Galois Counter Mode (GCM)
Offset Codebook Mode (OCB)
(more later)

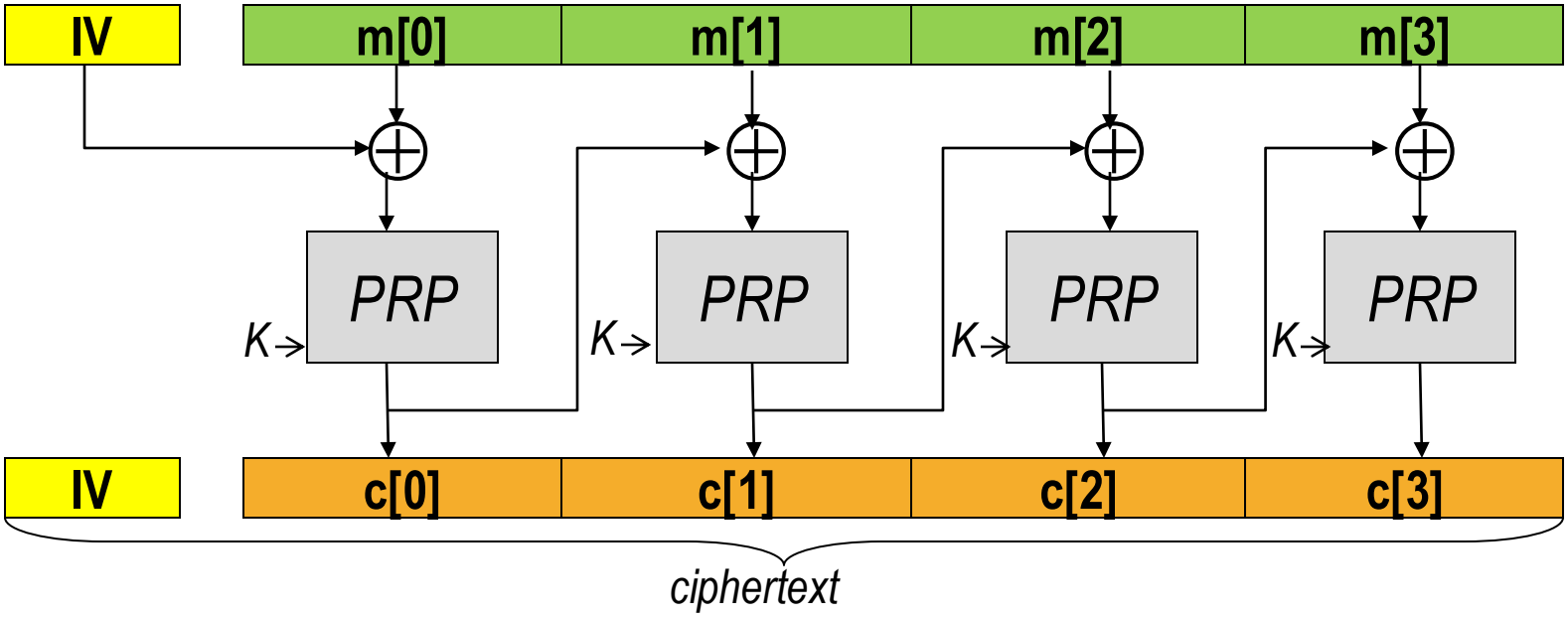
Semantic security: how to? (in principle)



If all iv(x) random → semantic security. But LOTS of overhead! (double size)

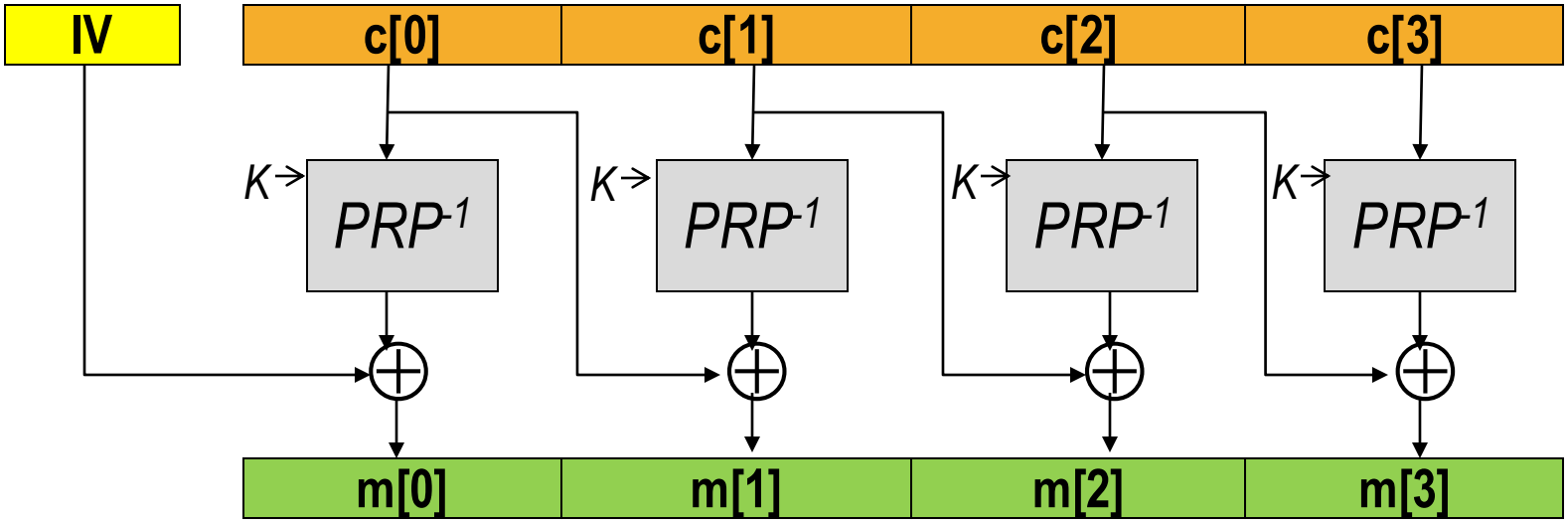
Cipher Block Chaining

- **Good PRP → pseudorandom output → c[i] «looks» as random**
- **CBC idea: use previous ciphertext block as IV for next block!!**
  - ⇨ *c[0] = ENC(K, IV ⊕ m[0])*
  - ⇨ *c[i] = ENC(K, c[i-1] ⊕ m[i])*



Ciphertext overhead: just the initial IV → marginal!

CBC decryption



➔CBC decryption: parallelizable!

⇨*Unlike CBC encryption, which has to be sequential*

➔not suitable for high speed HW implementation

CBC discussion

➔ **widely used (most common) & secure**

⇨ *But secure only if Nonce is random* **and non predictable**

→Predictable IV → CPA attack → exploited in TLS (2011, BEAST)

➔ **Slow during encryption**

⇨ *Non-parallelizable encryption → not appropriate for very high performance scenarios*

➔ **Encryption and decryption: require two different circuits**

⇨ *Encryption = PRP, decryption = PRP^{-1} → more circuitry*

⇨ *Other modes (CTR, CFB, OFB) use the PRP encryption block also in reception*

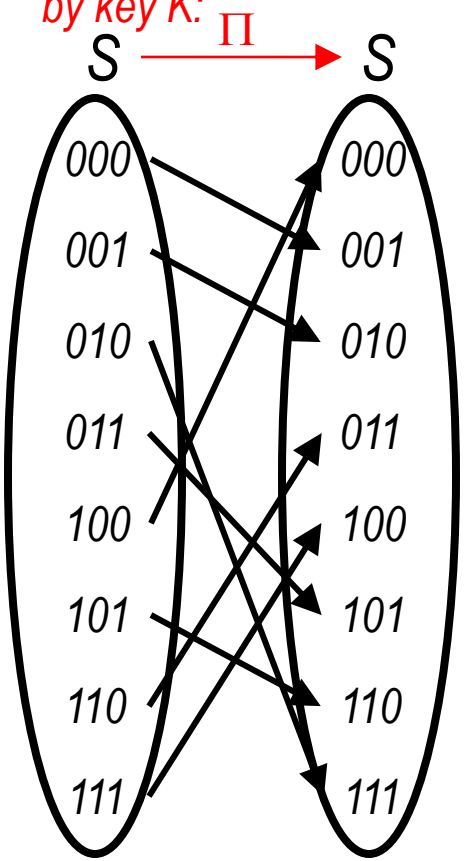
➔ **Plaintext must be multiple of block size**

⇨ *Padding used to «fill» space → standards do exist (e.g. PKCS#7)*

⇨ *Padding necessary also in (some) other modes, e.g. ECB*

Test your understanding

3-bit blocks;
Specific PRP selected by key K:



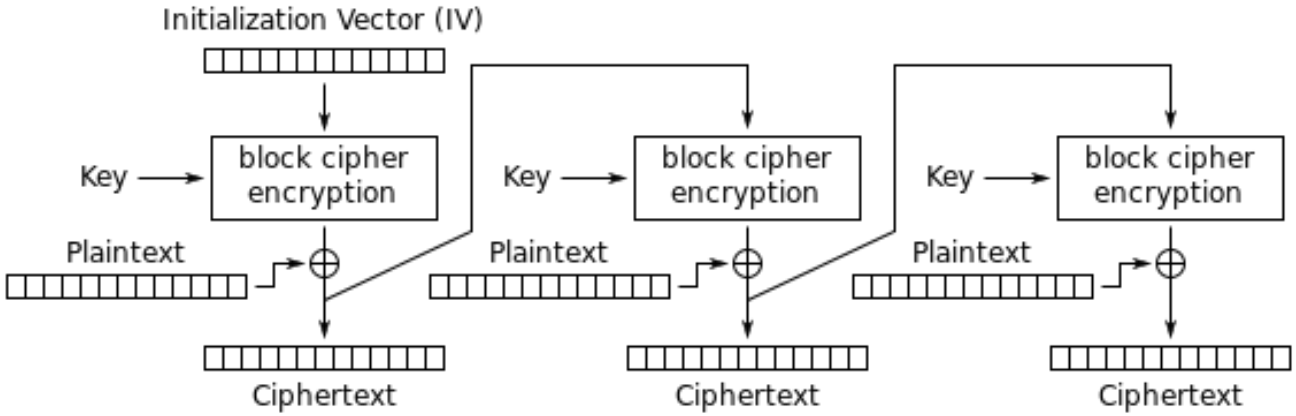
1) Encrypt M = 001 001 001 using ECB

CT = 010 010 010

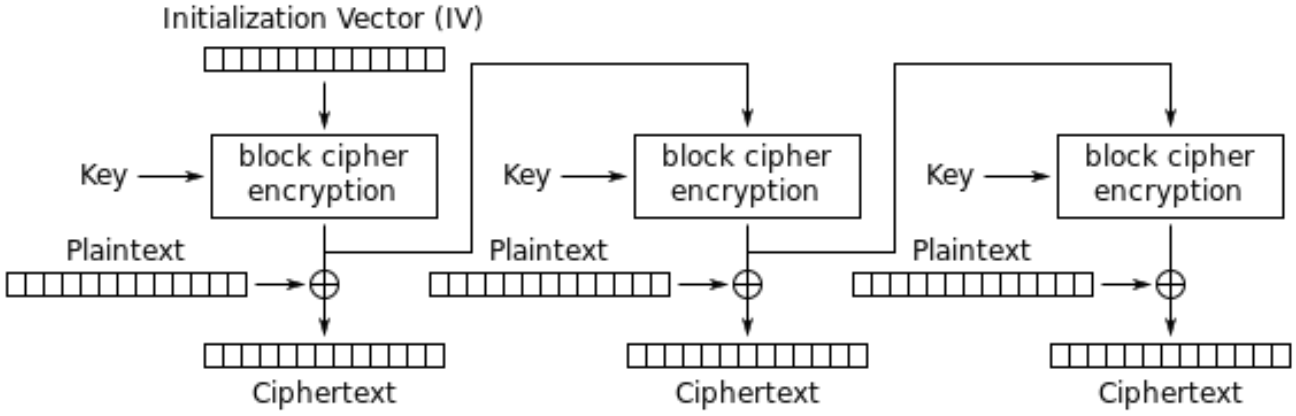
2) Encrypt M = 001 001 001 using CBC with IV=010

CT = (010) 101 000 010

Other modes: CFB, OFB



Cipher Feedback (CFB) mode encryption



Output Feedback (OFB) mode encryption

Block cipher used in «stream» manner

- *PT encrypted with XOR*
- *No need for padding*

Non parallelizable in encryption

- *But OFB advantage: permits preprocessing!*

Reason: CFB, like CBC, depends on previous ciphertexts

- $c[0] = m[0] \oplus ENC_K(IV)$
- $c[i] = m[0] \oplus ENC_K(c[i-1])$

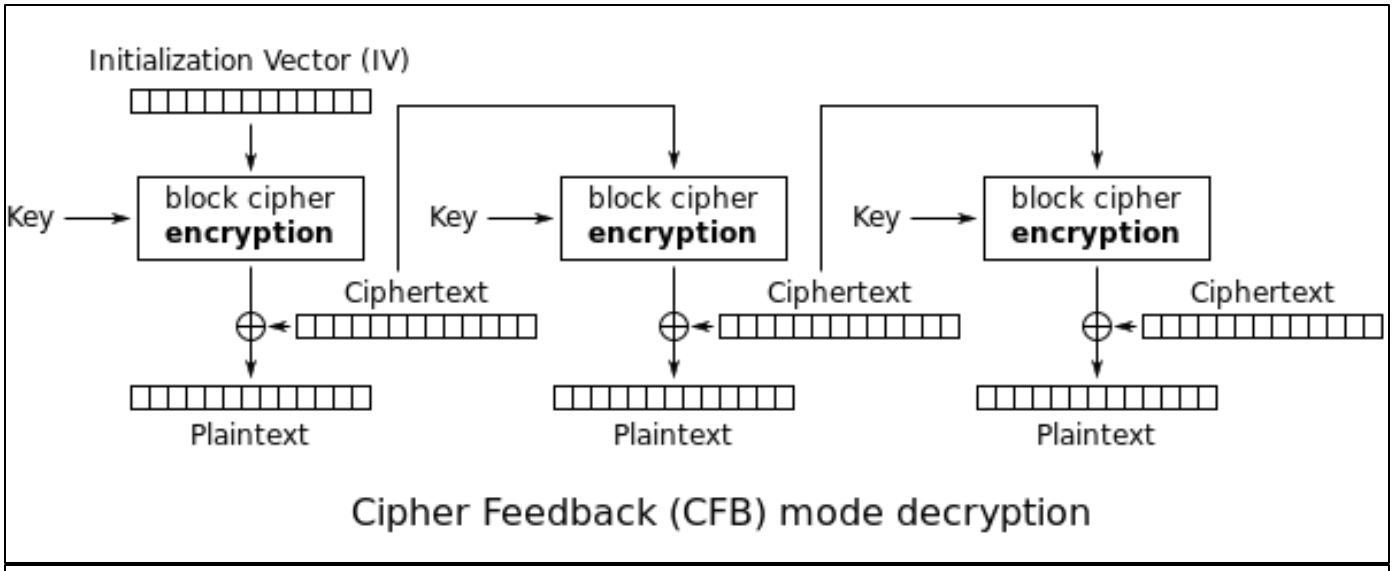
OFB «keystream» depends only on IV!

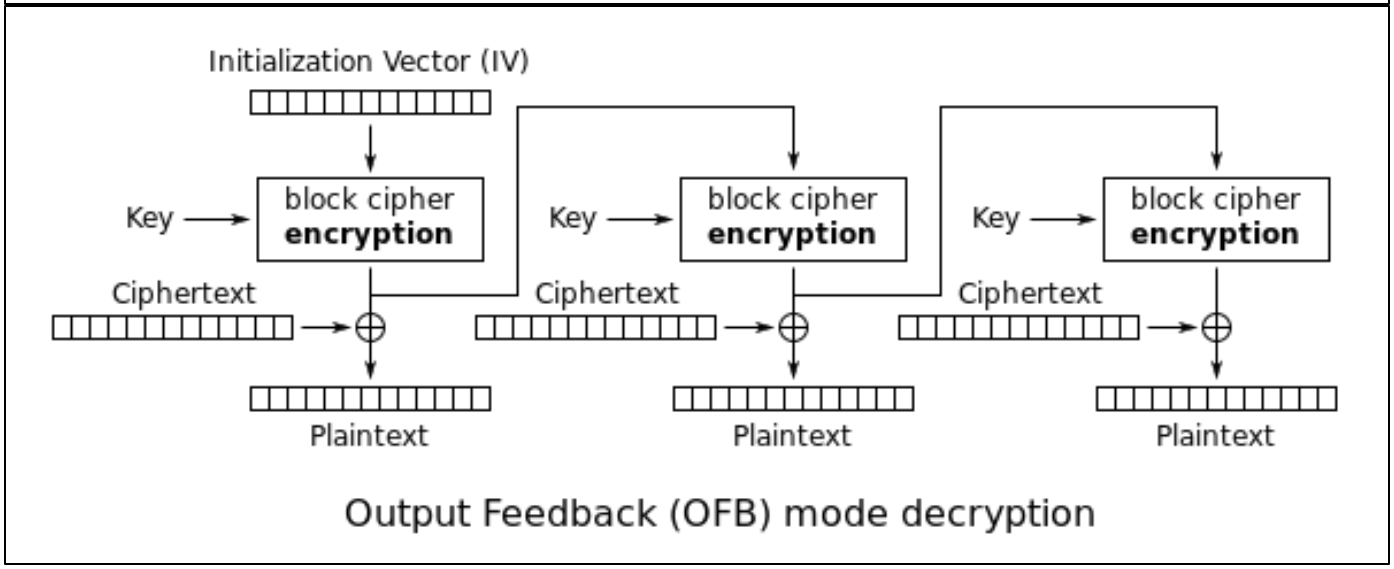
- **Robust to errors**

Looks like a classical stream cipher!

Graphics: source = Wikipedia

Other modes: CFB, OFB



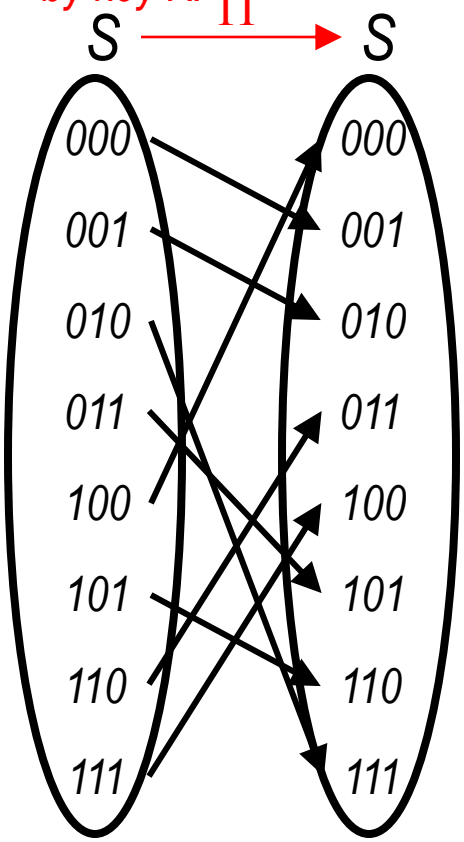


DECRYPTION advantage:
Reuse encryption block!
No need for «inverse» PRP implementation

Further CFB advantage:
Parallel decryption
OFB still serial decrypt

Chaining blocks: short cycle problem!

Example: 3-bit blocks; Specific PRP selected by key K:



1) OFB keystream, with IV=010?

010 111 100 000 001 010 111 …

cycle = 5

2) OFB keystream, with IV=011?

011 101 110 011 101 110 011 …

cycle = 3 – shorter! Cycle may depend on IV…

3) Same problem in CBC when encrypting a «regular» plaintext.

Example: encrypt 011 011 011 with IV=010

CT = (010) 010 010 010

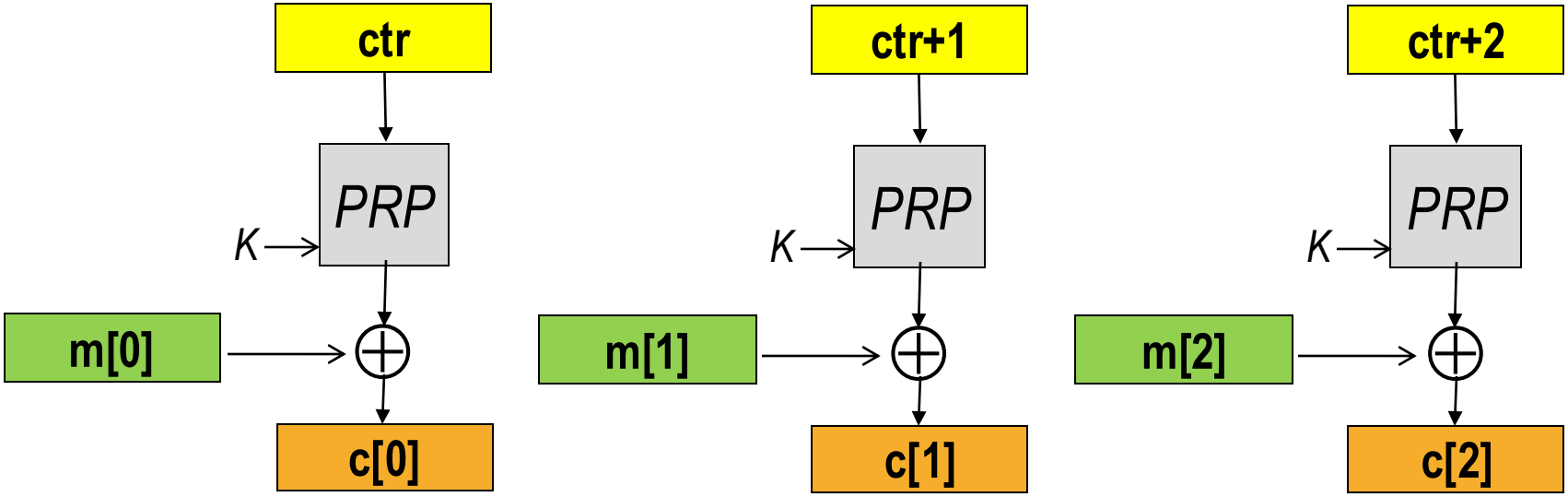
Counter Mode (CTR)

➔ **Very simple:**

- ⇨ *Initialize counter ctr, and increase it at every new block*
- ⇨ *«encrypt» counter with block cipher (independent of plaintext, can be precomputed)*
- ⇨ *XOR output with plaintext block ➔ looks like a keystream in stream cipher*

➔ **In pratice: builds a PRNG keystream out of a block PRP!**

- ⇨ *Provably secure: if PRP is secure then PRNG is secure as well*



AES-CTR detail on Initial Counter Block (ICB), see e.g. RFC3686:
first 96 bits = IV, next 32 bits = counter, starting from 1

Counter Mode (CTR)

- **Mode with most of the advantages!**
  - ⇨ *Turns block cipher into a stream one*
  - ⇨ *Combines all the advantages of CFB and OFB!*
- **Implementation efficiency (SW/HW)**
  - →Parallel encryption AND Parallel decryption
- **Requires implementation of encryption block only**
  - ⇨ *Unlike ECB and CBC which require also inverse block*
- **Random access**
  - ⇨ *Decryption of ith block does not depend on previous blocks*
    - →Unlike CBC or CFB, where you need to decrypt all previous blocks
- **Secure**
  - ⇨ *Counters (if properly used) do not repeat! Predictability NOT a concern!*
  - ⇨ *Guaranteed NO short cycle problems!*
    - →Since block is permutation, same output when same input repeats
      - » periodicity = 2^n

Authentication with Block ciphers: Which mode?

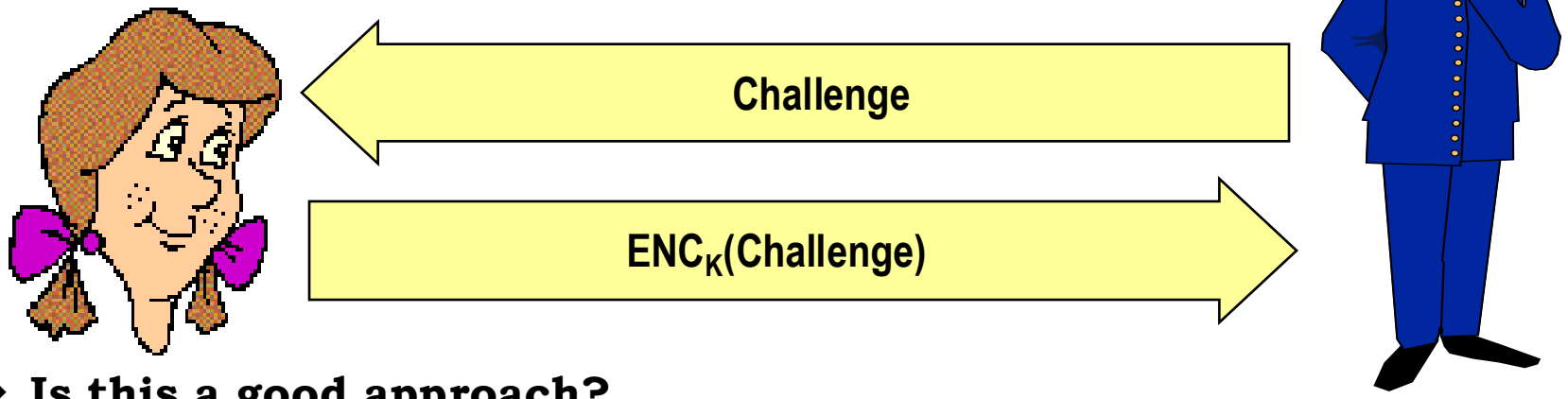
Back to CHAP-type authentication

➔ **How to "prove" I know a secret?**

➔ **Two practical approaches (no ZK today ☺)**

⇨ *Hash(secret, challenge)*

⇨ *ENC_{secret}(challenge)*



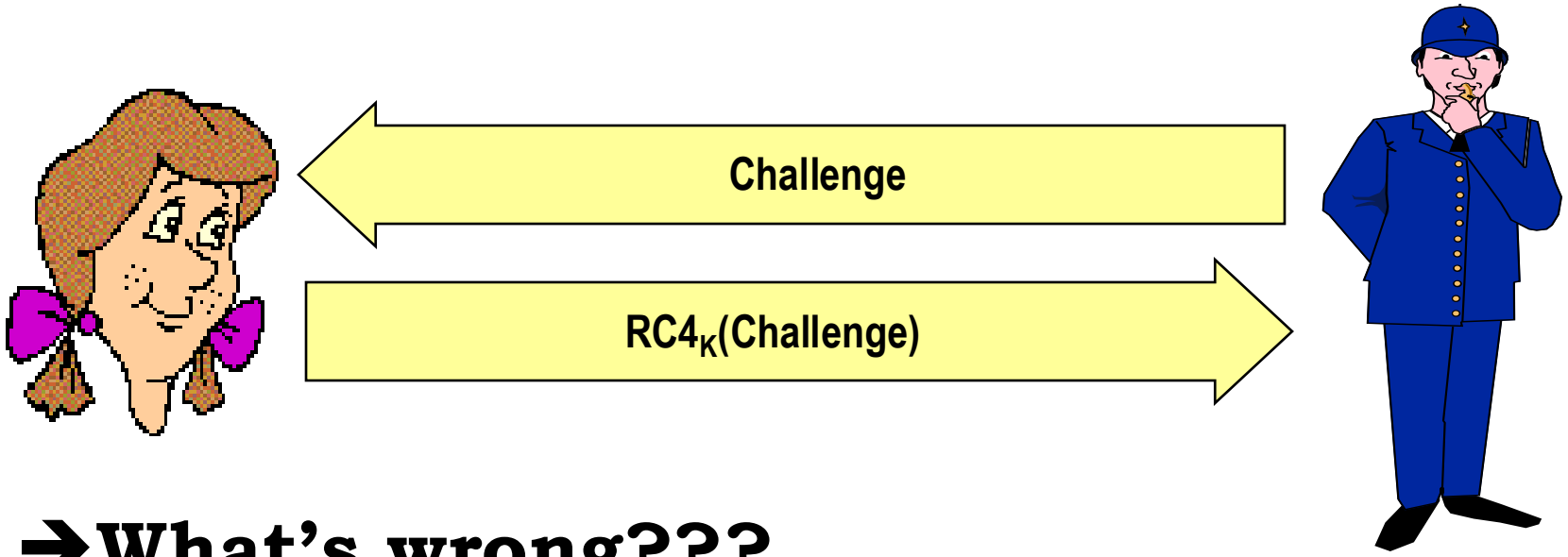
➔ **Is this a good approach?**

➔ **Let's check the Menezes handbook - https://cacr.uwaterloo.ca/hac/**

⇨ *Yes, chapter 10.3.2 mentions this*

⇨ *As long as challenge random and never repeats, I'm OK then*

WiFi WEP (badly broken) authentication



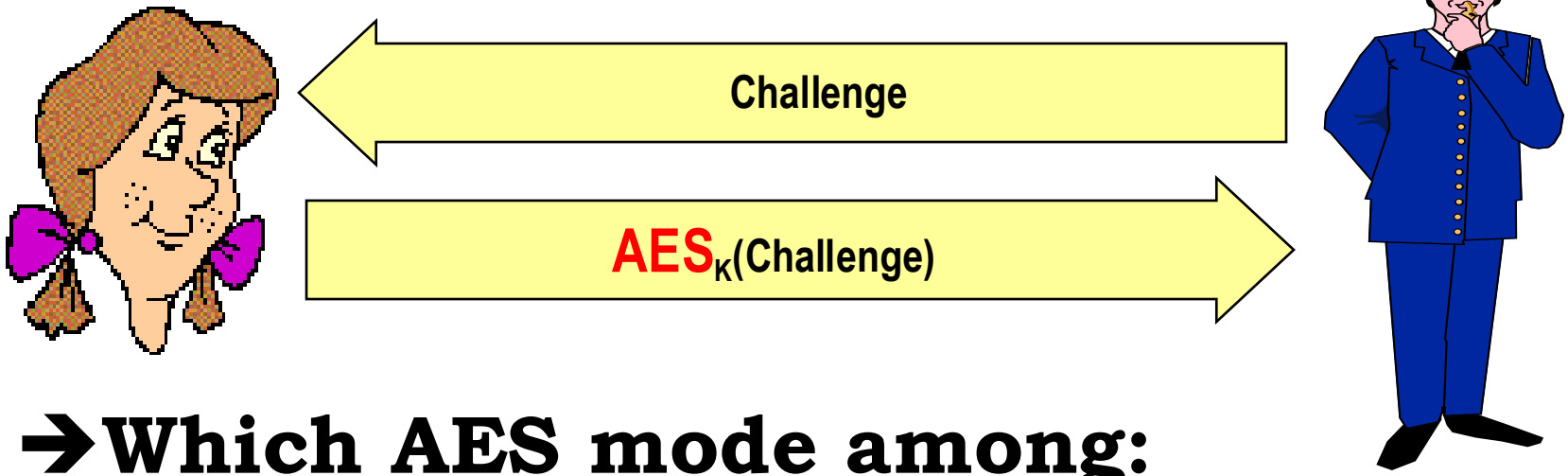
➔**What's wrong???**

➔**Ha Ha, RC4 is broken**



➔**Let's fix by using AES!!**

Let's replace (broken) RC4 with AES



➔Which AES mode among:

- ⇨*AES-ECB? (worst mode, insecure)*
- ⇨*AES-CBC? (ok-ish, but old-ish)*
- ⇨*AES-CTR? (best mode)*



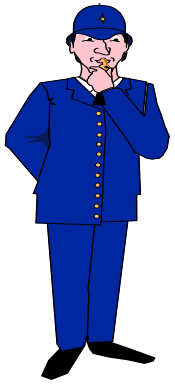
Let's recap WHY WEP auth was broken!

➔**Attacker sees:**



Challenge

IV, Challenge⊕RC4(IV,K)



➔**Extracts keystream RC4(IV,K)**

⇨*(a Known Plaintext Attack situation!)*

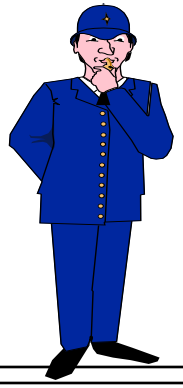


➔**And «encrypts» a NEW challenge by reusing previous IV and keystream!**



NewChal

IV, NewChal⊕RC4(IV,K)



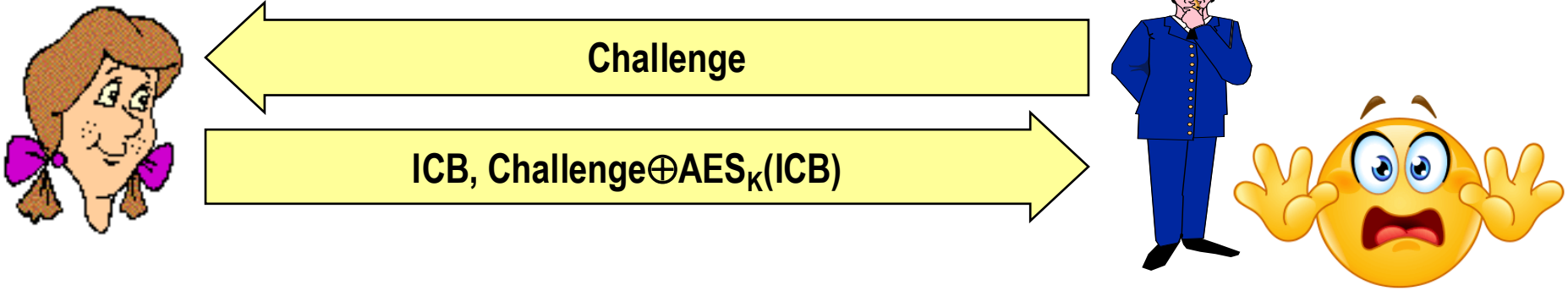
What if we use AES-CTR? Identical (!!) attack pattern!

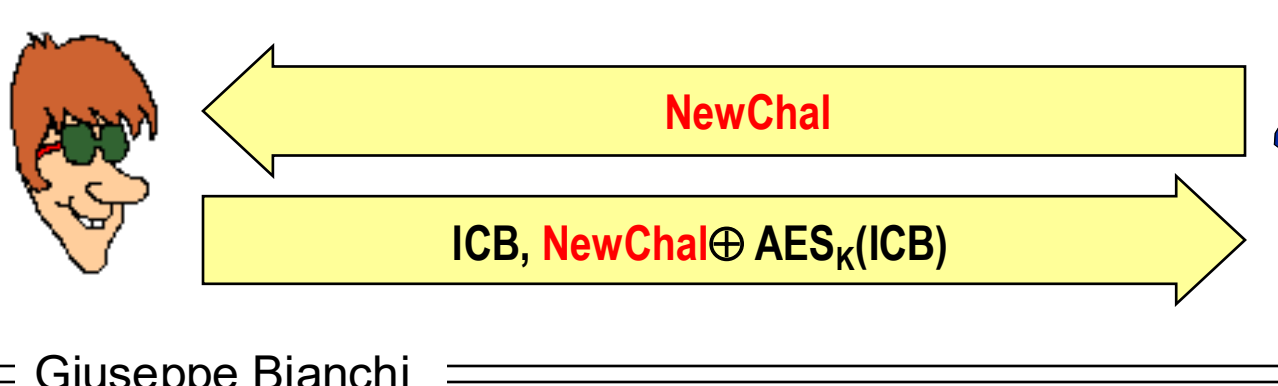
➔ **AES-CTR encryption also based on XOR:**

⇨ *Ciphertext = Initial counter, Challenge ⊕ AES_K(initial counter)*

➔ **Same attack!**

⇨ *NewCiphertext = Initial counter, Newchallenge ⊕ AES_K(initial counter)*





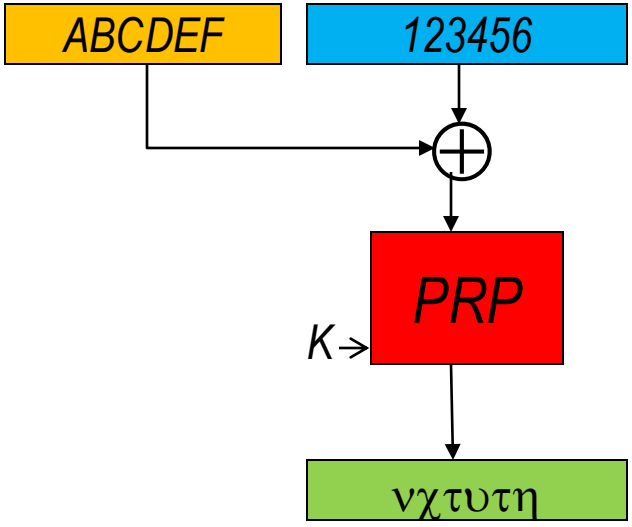
And what about AES-CBC?

Deadly, again!



IV chosen by victim

Original Challenge



Original response



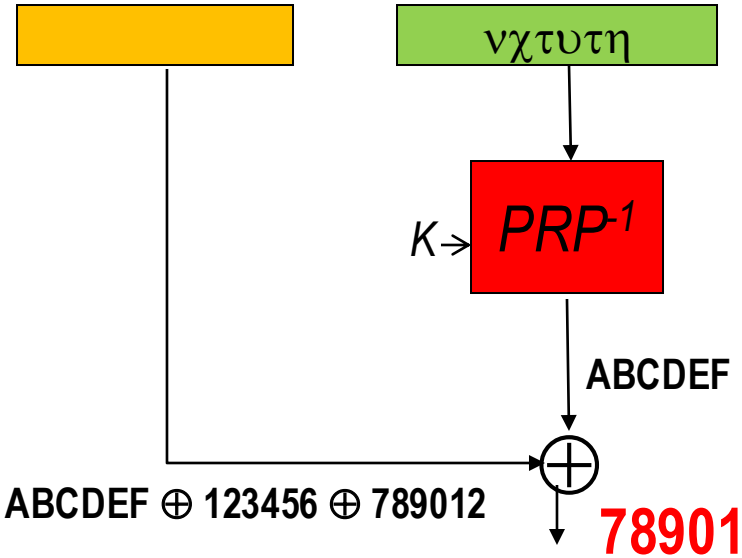
New challenge: 789012

Attacker responds with:

newIV = ABCDEF ⊕ 123456 ⊕ 789012

CT = replay previous CT = νχτυτη

Authenticator's validation:



789012 OK!

Are you saying the most secure mode is the worst one, AES-ECB?

➔ **Yes!!!!!!!!!!!!!!!!**

➔ **(remember, there is no best or worst, depends on attacker's model)**

⇨ *As long as Challenge never repeats, no way for attacker to forge a valid auth*

➔(no IV to replay or manipulate!)

Take home

1. We are using an ENC algo for a DIFFERENT purpose!

⇨ *The worst ENC algo can become the best now!*

2. Golden AUTH rule: leave ZERO freedom to authenticating device! And nonce (e,g, challenge) dediced by authenticator!

⇨ *Device much only respond to authenticator requests, with no extra parameters to «decide» (e.g. IV)*